Laboratorium 2 - Kamera, animacja i sterowanie w WebGL2

Celem zajęć jest rozbudowanie własnego projektu wykonanego na Laboratorium 1 o implementację kamery, pętli animacji, poruszających się samochodów oraz sterowania graczem. Laboratorium 2 jest kontynuacją Road Hoppera: należy zachować dodatkowe samochody, drzewa, pasy planszy, kolory i pozostałe samodzielnie wykonane modyfikacje.

Przygotowanie środowiska

1. Otwórz folder swojego projektu z Laboratorium 1. Zainstaluj bibliotekę **gl-matrix** zawierającą gotowe funkcje do tworzenia macierzy 4×4. Następnie ponownie uruchom aplikację:

npm install gl-matrix

npm run dev

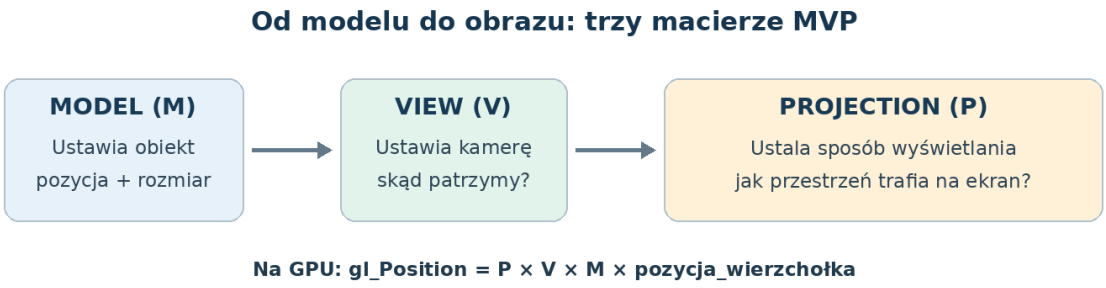
Zależność zostanie dopisana do **package.json**. W kolejnych krokach biblioteka **gl-matrix** zastąpi odręczne obliczenia projekcji z pierwszego laboratorium.

1. Kamera - po co potrzebujemy macierzy MVP?

Na poprzednim laboratorium położenie obiektów na ekranie wyznaczaliśmy za pomocą prostych obliczeń w Vertex Shaderze. Takie podejście pozwalało uzyskać uproszczony widok przestrzenny, jednak nie umożliwiało wygodnego sterowania kamerą, obracania obiektów ani stosowania różnych rodzajów perspektywy.

W grafice trójwymiarowej wykorzystujemy w tym celu macierze transformacji, czyli uporządkowane tablice liczb pozwalające przeliczać współrzędne wierzchołków. Najczęściej stosujemy macierze o wymiarach 4×4, składające się z 16 wartości. Dzięki nim możemy łączyć przesunięcie, obrót, skalowanie oraz operacje związane z kamerą.

W potoku MVP każdy wierzchołek przechodzi przez trzy transformacje:



1.1 Macierz Model (M) - transformacja obiektu

Macierz Model określa, w jaki sposób obiekt jest umieszczony w świecie gry. Pozwala zmieniać jego pozycję, rozmiar oraz obrót bez modyfikowania oryginalnych współrzędnych wierzchołków.

W przypadku skalowania i przesunięcia, bez obrotu, macierz przyjmuje postać:

$$M = \begin{bmatrix} s_x & 0 & 0 & t_x \\ 0 & s_y & 0 & t_y \\ 0 & 0 & s_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Gdzie s_x, s_y, s_z określają skalę obiektu, natomiast t_x, t_y, t_z jego przesunięcie względem początku układu współrzędnych. W tej postaci najpierw stosowane jest skalowanie, a następnie przesunięcie.

1.2 Macierz View (V) - położenie kamery

Macierz View odpowiada za sposób obserwowania sceny. Przelicza współrzędne obiektów ze świata gry do układu współrzędnych kamery. Przesunięcie kamery nie zmienia rzeczywistych pozycji samochodów, drzew ani gracza. Zmienia jedynie ich położenie względem obserwatora.
Dla kamery bez obrotu macierz View można przedstawić następująco:

$$V = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 1 & -c_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Wartości c_x, c_y, c_z oznaczają położenie kamery w świecie. Ujemne znaki wynikają z tego, że transformacja View przelicza świat względem kamery. Przykładowo, jeżeli przesuniemy kamerę w prawo, obiekty na ekranie przesuną się w lewo.

W praktyce macierz View uwzględnia także obrót kamery, np. poprzez funkcję lookAt(), która określa pozycję obserwatora, punkt obserwacji oraz kierunek uznawany za górę.

2.3. Macierz Projection (P) - sposób wyświetlania przestrzeni

Macierz Projection odpowiada za odwzorowanie przestrzeni widzianej przez kamerę na obszar obrazu. Określa m.in. zakres widoczności i sposób przedstawiania głębi.

W grafice 3D stosujemy przede wszystkim dwa rodzaje projekcji:

- **Projekcja ortograficzna** - Obiekty zachowują taki sam rozmiar niezależnie od odległości od kamery. Projekcja ta jest wykorzystywana m.in. w grach izometrycznych, mapach i projektowaniu CAD.

$$P_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & -\frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Gdzie: l - lewa granica, r - prawa granica, b - dolna granica, t- górna granica, n- odległość do bliższej płaszczyzny obcinania (near), f - odległość do dalszej płaszczyzny (far).

- **Projekcja perspektywiczna** - Obiekty oddalone od kamery wydają się mniejsze, natomiast obiekty położone bliżej są większe. Projekcja ta odwzorowuje sposób postrzegania głębi przez człowieka i jest powszechnie stosowana w grach 3D.

$$P_{persp} = \begin{bmatrix} \frac{1}{a\,tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{tan(\theta/2)} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Gdzie: a- proporcje ekranu (aspect ratio, szerokość/wysokość), θ - pionowy kąt widzenia kamery (Field of View), n i f-odległości do bliższej i dalszej płaszczyzny obcinania.

1.4 Zastosowanie w WebGL2

W naszym projekcie nie będziemy ręcznie obliczać wszystkich elementów macierzy 4×4. Wykorzystamy do tego bibliotekę **gl-matrix**, a do Vertex Shadera przekażemy gotowe macierze jako uniformy.

Wykonaj poniższe polecenia:

1. Otwórz *src/shaders/vertexShader.js*. Na potrzeby Lab 2 zastąp dotychczasową treść tego pliku poniższym kodem. Pozostałych shaderów nie zmieniaj.

```
export const vertexShaderSource = `#version 300 es
in vec3 aPosition;
uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;

void main() {
   gl_Position = uProjection * uView * uModel * vec4(aPosition, 1.0);
}`;
```

Dzięki takiemu rozwiązaniu każdy obiekt może mieć własną macierz Model, podczas gdy kamera i projekcja pozostają wspólne dla całej sceny. Zmiana pozycji kamery nie wymaga więc modyfikowania współrzędnych samochodów i drzew. Analogicznie, przesunięcie jednego samochodu nie wymaga zmiany ustawienia kamery ani pozostałych obiektów.
2. Otwórz *src/webgl/renderer.js*. Ta funkcja dotychczas przesyłała oddzielne uniformy pozycji i skali. Od teraz połączymy je w jedną macierz Model. Zastąp zawartość pliku następującym kodem:

```
import { mat4 } from 'gl-matrix';

export function drawMesh(gl, mesh, uniforms, object) {
   gl.bindVertexArray(mesh.vao);

   const model = mat4.create();
   mat4.translate(model, model, object.position);
   mat4.scale(model, model, object.scale);

   gl.uniformMatrix4fv(uniforms.model, false, model);
   gl.uniform3fv(uniforms.color, object.color);
   gl.drawArrays(gl.TRIANGLES, 0, mesh.vertexCount);
}
```

Macierz Model najpierw uwzględnia skalę, a następnie pozycję bryły w świecie. Wykorzystujemy te same object.position i object.scale, co w poprzednim laboratorium.

3. Otwórz *src/main.js* i wprowadź trzy niewielkie zmiany. Na początku dodaj import:

```
import { mat4 } from 'gl-matrix';
```

Następnie odszukaj obiekt uniforms i zastąp wyłącznie jego definicję. Nazwy powinny odpowiadać nowym uniformom z Vertex Shadera:

```
const uniforms = {
   model: gl.getUniformLocation(program, 'uModel'),
   view: gl.getUniformLocation(program, 'uView'),
   projection: gl.getUniformLocation(program, 'uProjection'),
   color: gl.getUniformLocation(program, 'uColor')
};
```

Poniżej utworzonych *cubeMesh* i *planeMesh* dodaj ustawienia kamery:

```
const camera = {
   eye: [2.5, 2.8, -3.8],      // Gdzie stoi kamera?
   target: [0.0, -0.15, -0.35], // Na co patrzy?
   up: [0, 1, 0],              // Gdzie jest góra obrazu?
   zoom: 1.8                   // Zakres pokazywanego obszaru
};
```

4. W funkcji *render()* znajdź i usuń tylko polecenie wysyłające stary uniform uAspect (wywołanie *gl.uniform1f(uniforms.aspect, ...)*). W jego miejsce dodaj blok:

```
const aspect = canvas.width / canvas.height;

const view = mat4.create();
mat4.lookAt(view, camera.eye, camera.target, camera.up);

const projection = mat4.create();
mat4.ortho(projection,
    -camera.zoom * aspect, camera.zoom * aspect,
    -camera.zoom, camera.zoom, 0.1, 100);

gl.uniformMatrix4fv(uniforms.view, false, view);
gl.uniformMatrix4fv(uniforms.projection, false, projection);
```

Po zapisaniu zmian i odświeżeniu powinieneś zobaczyć swoją dotychczasową scenę, ale z nowej perspektywy kamery. Jeśli ekran jest pusty, sprawdź w konsoli, czy nazwy *uModel*, *uView* i *uProjection* są zgodne we wszystkich trzech miejscach: shaderze, rendererze i *main.js*.

Eksperyment 1. Inny kierunek patrzenia

Zmień wyłącznie camera.eye z [2.5, 2.8, -3.8] na [0.0, 4.5, -3.0]. Sprawdź różnicę. Samochody i drzewa nie zmieniają położenia w świecie - przesunęła się kamera, nie obiekty. Przywróć oryginalną wartość. Następnie zmień zoom: 1.8 na zoom: 2.4 i zaobserwuj, że widać szerszy fragment planszy. Przywróć 1.8.

3. Animacja – dlaczego jedna klatka nie wystarcza?

Do tej pory po uruchomieniu programu obraz był rysowany tylko raz. Aby samochód się poruszał, trzeba regularnie aktualizować jego pozycję i ponownie rysować obraz. requestAnimationFrame() zleca przeglądarce wywołanie funkcji przed następną klatką.



1. Przygotuj miejsce na logikę gry. W src/game utwórz nowy plik *gameLogic.js* z poniższą funkcją. Na razie niczego nie porusza, ale umożliwi podłączenie pętli animacji.

```
export function updateCars(dt) {
   // W następnym etapie uzupełnimy ruch samochodów. }
```

2. W *src/main.js* dodaj import nowej funkcji na początku pliku:

```
import { updateCars } from './game/gameLogic.js';
```

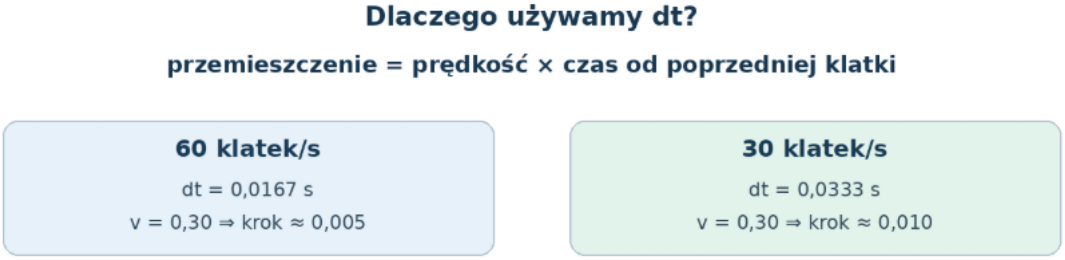
3.Na dole *main.js* zastąp jednorazowe wywołanie *render()*; nowym kodem.

```
let previousTime = null;
function frame(time) {
   const dt = previousTime === null
      ? 0
      : Math.min((time - previousTime) / 1000, 0.05);
   previousTime = time;
   updateCars(dt);
   render();
 requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

Funkcja *frame()* będzie wywoływana wielokrotnie. time to czas w milisekundach, a dt jest czasem od poprzedniej klatki wyrażonym w sekundach. Dzięki temu będziemy mogli określać prędkość w jednostkach świata na sekundę, zamiast przesuwać samochód o stałą liczbę jednostek w każdej klatce. Po zapisaniu zmian samochody się nie poruszają, ponieważ funkcja *updateCars()* jest pusta, natomiast pętla już działa.

4. Ruch samochodów - prędkość i czas `dt`

W animacji ruch możemy wyrazić bardzo prosto: przesunięcie = prędkość × czas. Jeżeli samochód ma prędkość 0.4 jednostki świata na sekundę i od ostatniej klatki minęło 0.02 sekundy, przesunie się o 0.008 jednostki. Dzięki temu liczba klatek ma mniejszy wpływ na prędkość ruchu.



1. Otwórz własny *src/game/sceneData.js*. Do każdego obiektu w tablicy scene.cars dodaj pole speed. Nie nadpisuj dotychczasowych współrzędnych ani kolorów; przykładowo pierwszy samochód może otrzymać:

```
{
   position: [-0.60, 0.0, 0.0],
   color: [0.92, 0.12, 0.13],
   speed: 0.32
}
```

Dla drugiego samochodu ustaw np. speed: -0.42, dla trzeciego speed: 0.25. Jeśli masz więcej samochodów, każdemu również przypisz prędkość. Dodatnia wartość przesuwa pojazd w dodatnim kierunku osi X, a ujemna w przeciwnym. Pozostałe dane sceny - gracz, pasy i drzewa - pozostaw bez zmian.

2. W pliku *src/game/gameLogic.js* zamień pustą funkcję *updateCars()* na poniższą wersję:

```
import { scene } from './sceneData.js';
export function updateCars(dt) {
   const edge = 1.25;
   for (const car of scene.cars) {
      car.position[0] += car.speed * dt;
      if (car.speed > 0 && car.position[0] > edge) {
         car.position[0] = -edge;
      } else if (car.speed < 0 && car.position[0] < -edge) {
         car.position[0] = edge;
      }
   }
}
```

Pierwsza instrukcja zmienia pozycję samochodu w osi X. Dwa warunki sprawdzają, czy pojazd opuścił obszar gry, i przenoszą go na przeciwną stronę. To prosta animacja bez wykonywania nowych operacji na geometrii VBO. Zapisz zmiany i obejrzyj ruch samochodów.

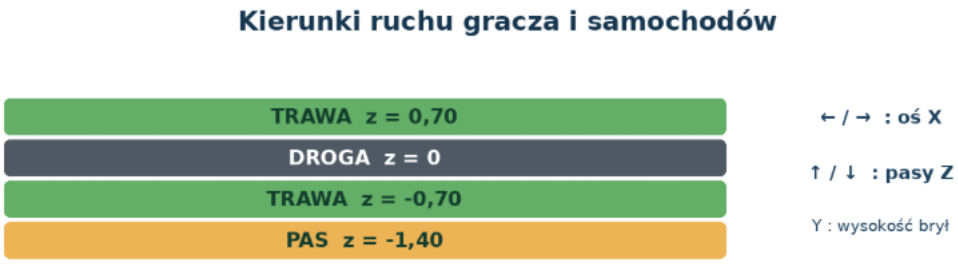
Eksperyment 2. Prędkość i kierunek ruchu

Dla jednego samochodu ustaw na chwilę speed: 0.8, następnie speed: -0.8 i na końcu speed: 0. Zwróć uwagę na szybkość oraz kierunek ruchu. Przywróć wartości początkowe. Następnie na kilka sekund zakomentuj w frame() wywołanie updateCars(dt) - samochody zatrzymają się, mimo że scena w dalszym ciągu jest renderowana. Przywróć wywołanie.

5. Sterowanie graczem – reakcja na klawisze

W grze typu Crossy Road samochody poruszają się w sposób ciągły i niezależny od działań użytkownika, natomiast ruchem postaci steruje gracz. Każde naciśnięcie odpowiedniego klawisza strzałki powinno powodować przesunięcie postaci o jeden krok w wybranym kierunku. W tym celu wykorzystujemy zdarzenie keydown, które wykrywa naciśnięty klawisz i przekazuje tę informację do logiki gry. Następnie funkcja odpowiedzialna za sterowanie aktualizuje pozycję gracza, a istniejąca pętla animacji wyświetla zmianę na ekranie. Ruch

postaci realizujemy poprzez modyfikację jej współrzędnych w świecie gry, bez konieczności wprowadzania zmian w shaderach.



1. W *src/game/gameLogic.js* zmień import na taki, który pobiera także pasy:

```
import { rows, scene } from './sceneData.js';
```

2.Pod istniejącą funkcją *updateCars()* dopisz funkcję sterowania graczem:

```
export function movePlayer(key) {
   const p = scene.player.position;
   const stepX = 0.25;
   const limitX = 0.75;

   if (key === 'ArrowLeft')  p[0] = Math.max(-limitX, p[0] - stepX);
   if (key === 'ArrowRight') p[0] = Math.min( limitX, p[0] + stepX);

   const lanes = rows.map(row => row.z).sort((a, b) => b - a);
   const index = lanes.findIndex(z => Math.abs(z - p[2]) < 0.001);
   if (key === 'ArrowUp' && index >= 0 && index < lanes.length - 1) {
      p[2] = lanes[index + 1];
   }
   if (key === 'ArrowDown' && index > 0) {
      p[2] = lanes[index - 1];
   }
}
```

Ruch w lewo i prawo dodaje lub odejmuje krok od współrzędnej X. Funkcje Math.*min()* i Math.*max()* zapobiegają wyjściu poza szerokość planszy. Strzałki pionowe przełączają gracza między poziomami Z występującymi w Twojej własnej tablicy `rows`, dlatego funkcja zadziała także wtedy, gdy masz dodatkowy pas z zadania Lab 1.

3. W src/main.js zmień import na:

```
import { updateCars, movePlayer } from './game/gameLogic.js';
```

Na końcu pliku, po zarejestrowaniu pętli animacji, dodaj obsługę klawiatury:

```
window.addEventListener('keydown', event => {
   if (!event.key.startsWith('Arrow')) return;
   event.preventDefault();
   if (event.repeat) return;
   movePlayer(event.key);
});
```

Funkcja event.repeat informuje, że klawisz pozostaje przytrzymany; jego pomijanie powoduje, że ruch następuje po pojedynczym naciśnięciu. Uruchom grę i spróbuj przesuwać gracza po wszystkich pasach.

Eksperyment 3. Wielkość kroku i granice planszy

1. Zmień chwilowo stepX z 0.25 na 0.10 i obserwuj, jak zmniejsza się odległość przebyta po jednym naciśnięciu. Przywróć 0.25. Następnie wielokrotnie naciśnij strzałki przy krawędzi planszy. Wyjaśnij, dlaczego postać nie wychodzi poza ograniczenia w osi X ani poza listę dostępnych pasów.

Zadania do sprawozdania

Po wykonaniu ćwiczeń samodzielnie rozbuduj swoją wersję projektu.

Zadanie 1. Zmień ustawienie kamery, aby obserwować planszę z perspektywy trzeciej osoby. Postaraj się zachować widoczność gracza, samochodów i wszystkich pasów. Do sprawozdania dołącz zrzuty ekranu przed i po zmianie.

Zadanie 2. Rozbuduj istniejącą drogę tak, aby składała się z dwóch pasów ruchu, po których samochody poruszają się w przeciwnych kierunkach. Każdy pas powinien mieć co najmniej dwa pojazdy poruszające się z różnymi prędkościami. Dodaj wizualne rozdzielenie pasów (np. przerywaną linię na środku drogi). Po opuszczeniu widocznego obszaru samochody powinny pojawiać się ponownie po przeciwnej stronie.

Zadanie 3. Dodaj możliwość zatrzymania i wznowienia ruchu samochodów przy użyciu wybranego klawisza. Zatrzymane pojazdy mają pozostać w aktualnych miejscach, a ich ponowne uruchomienie nie może powodować nagłego przeskoku. Sterowanie graczem powinno być nie możliwe.

Wymagania dotyczące sprawozdania

Sprawozdanie powinno być podzielone na poszczególne laboratoria, z wyraźnym oznaczeniem **numeru laboratorium oraz numerów wykonanych zadań**.

Dla każdego zadania należy zamieścić:

1. **Opis rozwiązania** - krótkie wyjaśnienie, w jaki sposób wykonano zadanie, np. poprzez przedstawienie zmodyfikowanego fragmentu kodu wraz z krótkim komentarzem.
2. **Zrzut ekranu**- przedstawiający efekt wprowadzonych zmian na renderowanej scenie.

Po zakończeniu trzech laboratoriów z WebGL2 należy przesłać **jedno zbiorcze sprawozdanie**, zawierające rozwiązania zadań ze wszystkich trzech spotkań.